

# Universidad Nacional Autónoma de México
## Facultad de Ingeniería



# Práctica No.05
# *Algoritmo de Cifrado Hill*

**Asignatura:**

CRIPTOGRAFÍA

**Profesor:** *Dr. Alfonso Francisco De Abiega L'Eglisse*

**Grupo:** 08

**Integrantes:**

Espinoza Matamoros Percival Ulises
García Cortés Adolfo de Jesús
Lugo Manzano Rodrigo
Montiel Juárez Oscar Iván

**Semestre:** 2027-1

# Práctica 05. Algoritmo de Cifrado: Hill

## 1. Explicación del Método

### 1.1. Letras como números

Se usa el alfabeto de 26 letras: `a`=0, `b`=1, ..., `z`=25. Todas las cuentas se hacen en **módulo 26**.

### 1.2. Cifrado

La clave es una matriz cuadrada $K$ de $2 \times 2$ o de $3 \times 3$. El mensaje se parte en bloques del mismo tamaño que la matriz (2 o 3 letras). Cada bloque es un vector columna $P$ y se cifra así:

$$C = K \cdot P \pmod{26}$$

Si al final el último bloque queda incompleto, se rellena con la letra `x`.

### 1.3. Descifrado

Para descifrar se necesita la **matriz inversa** de $K$ en módulo 26:

$$P = K^{-1} \cdot C \pmod{26}, \qquad K^{-1} = (\det K)^{-1} \cdot \operatorname{adj}(K) \pmod{26}$$

Aquí $(\det K)^{-1}$ es el inverso de $\det K$ en módulo 26, es decir, el número $x$ tal que $\det K \cdot x \equiv 1 \pmod{26}$.

### 1.4. Cuándo sirve una clave

La matriz solo tiene inversa en módulo 26 si $\gcd(\det K, 26) = 1$. Es decir, el determinante no puede ser par ni múltiplo de 13. Si no se cumple, no se podría descifrar el mensaje, y por eso el programa rechaza esas claves.

### 1.5. Ejemplo con la clave `dcdf`

La palabra `dcdf` son los números 3, 2, 3, 5. Se llena la matriz por filas:

$$K = \begin{pmatrix} 3 & 2 \\ 3 & 5 \end{pmatrix}, \qquad \det K = 3 \cdot 5 - 2 \cdot 3 = 9$$

Como $\gcd(9, 26) = 1$, la clave sirve. El inverso de 9 en módulo 26 es 3, porque $9 \cdot 3 = 27 \equiv 1$. La matriz inversa queda

$$K^{-1} = \begin{pmatrix} 15 & 20 \\ 17 & 9 \end{pmatrix}$$

Para el bloque `ho` $= (7, 14)$:

$$\begin{pmatrix} 3 & 2 \\ 3 & 5 \end{pmatrix} \begin{pmatrix} 7 \\ 14 \end{pmatrix} = \begin{pmatrix} 49 \\ 91 \end{pmatrix} \equiv \begin{pmatrix} 23 \\ 13 \end{pmatrix} \pmod{26}$$

Los números 23 y 13 son las letras `x` y `n`. Entonces `ho` se cifra como `xn`.

# 2. Implementación en Python

## 2.1. Lectura de la clave y creación de la matriz

```python
def leer_clave(entrada):

    # Convierte la clave escrita por el usuario en una lista de números (0-25).
    # Acepta una palabra (como 'dcdf') o números separados por espacio (como '3 2 3 5').
    ## De igual forma no regresamos nada si no lo podemos descifrar el mensaje

    partes = entrada.split()

    if len(partes) == 1 and all(c in A for c in partes[0]):
        numeros = [A.index(c) for c in partes[0]]      # clave en letras
    else:
        try:
            numeros = [int(p) % N for p in partes]      # clave en números
        except ValueError:
            return None

    return numeros if len(numeros) in (4, 9) else None


def matriz_desde_clave(numeros):

    # Convierte la lista de números en matriz, llenando por filas.
    # 4 valores -> 2x2, 9 valores -> 3x3. Ej: [3, 2, 3, 5] -> [[3, 2], [3, 5]]

    n = 2 if len(numeros) == 4 else 3
    return [numeros[i * n:(i + 1) * n] for i in range(n)]
```

La clave puede escribirse como una palabra (de 4 o 9 letras) o como números separados por espacios (4 o 9 números). Con 4 valores se arma una matriz de $2 \times 2$ y con 9 valores una de $3 \times 3$, llenando por filas. Si la clave no tiene un formato válido, la función regresa `None` y el programa vuelve a pedirla.

## 2.2. Determinante, adjunta e inversa

```python
def determinante(K):
    # Determinante de una matriz 2x2 o 3x3. Se asume que K es cuadrada y de tamaño 2 o 3.
    if len(K) == 2:
        return K[0][0] * K[1][1] - K[0][1] * K[1][0]
    a, b, c = K[0]
    d, e, f = K[1]
    g, h, i = K[2]
    return a * (e * i - f * h) - b * (d * i - f * g) + c * (d * h - e * g)

```

```
10
11  def adjunta(K):
12      # Matriz adjunta
13      n = len(K)
14      if n == 2:
15          return [[K[1][1], -K[0][1]], [-K[1][0], K[0][0]]]
16
17      adj = [[0] * 3 for _ in range(3)]
18      for i in range(3):
19          for j in range(3):
20              # Menor: matriz 2x2 que queda al quitar la fila i y la columna
                    j
21              menor = [[K[r][c] for c in range(3) if c != j] for r in range
                    (3) if r != i]
22              cofactor = ((-1) ** (i + j)) * determinante(menor)
23              adj[j][i] = cofactor
24      return adj
25
26
27  def matriz_inversa(K):
28      # K^-1 = (det K)^-1 * adj(K) mod 26
29      det_inv = pow(determinante(K) % N, -1, N)
30      adj = adjunta(K)
31      n = len(K)
32      return [[(det_inv * adj[i][j]) % N for j in range(n)] for i in range(n
            )]
```

Se calcularon a mano estas funciones, sin usar librerías externas:

- `determinante` usa la fórmula directa para $2 \times 2$ y el desarrollo por la primera fila para $3 \times 3$.
- `adjunta` calcula los cofactores (con el menor de cada posición) y los pone transpuestos.
- `matriz_inversa` multiplica la adjunta por el inverso del determinante. El inverso se obtiene con `pow(det, -1, 26)`.

## 2.3. Funciones de cifrado y descifrado

```
1   def aplicar_matriz(texto, K, mostrar_detalle=False, etiqueta="P"):
2
3       # Multiplica cada bloque (vector columna) por la matriz K mod 26.
4       # Se usa para cifrar (con K) y para descifrar (con K^-1).
5
6       n = len(K)
7       resultado = ""
8
9       for i in range(0, len(texto), n):
10          bloque = texto[i:i + n]
11          x = [A.index(c) for c in bloque]
12          y = [sum(K[f][c] * x[c] for c in range(n)) % N for f in range(n)]
13          letras = "".join(A[v] for v in y)
14
```

```
        if mostrar_detalle:
            print(f"'{bloque}' ({etiqueta}={x}) -> {y} -> '{letras}'")

        resultado += letras

    return resultado


def cifrar(mensaje, K, mostrar_detalle=False):
    n = len(K)
    # Quitamos espacios y rellenamos con 'x' hasta completar el último bloque
    texto = mensaje.replace(" ", "")
    while len(texto) % n != 0:
        texto += "x"

    # Fórmula: C = K * P mod 26
    return aplicar_matriz(texto, K, mostrar_detalle, "P")


def descifrar(mensaje_cifrado, K, mostrar_detalle=False):
    # Fórmula: P = K^-1 * C mod 26
    return aplicar_matriz(mensaje_cifrado, matriz_inversa(K), mostrar_detalle, "C")
```

La función `aplicar_matriz` multiplica cada bloque por la matriz y saca el módulo 26. Se usa tanto para cifrar (con $K$) como para descifrar (con $K^{-1}$), así que solo cambia la matriz que se le manda. En `cifrar` se quitan los espacios y se agregan `x` hasta completar el último bloque.

## 2.4. Validación de datos

```
# Validación de la clave: palabra o números, 4 valores (2x2) o 9 valores (3x3)
while True:
    clave = input("> Ingrese la clave (palabra o números separados por espacio; 4 para 2x2, 9 para 3x3): ").lower()

    numeros = leer_clave(clave)
    if numeros is None:
        print("\n![Error]: Ingrese una palabra de 4 o 9 letras (a-z) o 4 o 9 números enteros separados por espacio.\n")
        continue

    K = matriz_desde_clave(numeros)
    det = determinante(K) % N

    # La matriz es invertible mod 26 solo si minimo comun Divisor común mas grande(det, 26) = 1
    if gcd(det, N) == 1:
        break
    else:
```

```
Problems 1   Output   Debug Console   Terminal   Ports
PS C:\Users\adolf\AppData\Local\Programs\Microsoft VS Code> & 'C:\Users\adolf\AppData\Local\Programs\Python\Python311\python.exe'
.py"'

  Algoritmo de Cifrado Hill

> Ingrese el mensaje a cifrar: hola equipo diez
> Ingrese la clave (palabra o números separados por espacio; 4 para 2x2, 9 para 3x3): dcdf

--- Cálculo del cifrado (C = K * P mod 26) ---
'ho' (P=[7, 14]) -> [23, 13] -> 'xn'
'la' (P=[11, 0]) -> [7, 7] -> 'hh'
'eq' (P=[4, 16]) -> [18, 14] -> 'so'
'ui' (P=[20, 8]) -> [24, 22] -> 'yw'
'po' (P=[15, 14]) -> [21, 11] -> 'vl'
'di' (P=[3, 8]) -> [25, 23] -> 'zx'
'ez' (P=[4, 25]) -> [10, 7] -> 'kh'

--- Cálculo del descifrado (P = K^-1 * C mod 26) ---
'xn' (C=[23, 13]) -> [7, 14] -> 'ho'
'hh' (C=[7, 7]) -> [11, 0] -> 'la'
'so' (C=[18, 14]) -> [4, 16] -> 'eq'
'yw' (C=[24, 22]) -> [20, 8] -> 'ui'
'vl' (C=[21, 11]) -> [15, 14] -> 'po'
'zx' (C=[25, 23]) -> [3, 8] -> 'di'
'kh' (C=[10, 7]) -> [4, 25] -> 'ez'

------------ Resultados ------------

--> Clave utilizada:          dcdf
--> Matriz clave K:           [[3, 2], [3, 5]]
--> Determinante (mod 26):    9
--> Matriz inversa K^-1:      [[15, 20], [17, 9]]
--> Mensaje original:         hola equipo diez
--> Mensaje cifrado:          xnhhsoywvlzxkh
--> Mensaje descifrado:       holaequipodiez

--------------------------------------
```

Figura 1: Prueba 1

```
17          print(f"\n![Error]: El determinante ({det}) no es coprimo con 26,
                la matriz no tiene inversa.\n")
18
19  K_inv = matriz_inversa(K)
```

Aquí se comprueba que la matriz tenga inversa: si el máximo común divisor entre el determinante y 26 no es 1, el programa muestra un error y pide otra clave.

# 3. Pruebas del Programa

## 3.1. Prueba 1: matriz $2 \times 2$ con la clave `dcdf`

El mensaje “hola equipo diez” tiene 14 letras sin espacios, así que forma 7 bloques de 2 letras exactos y no hace falta relleno. El resultado cifrado es `xnhhsoywvlzxkh`.

```
  Algoritmo de Cifrado Hill

> Ingrese el mensaje a cifrar: ataque al amanecer
> Ingrese la clave (palabra o números separados por espacio; 4 para 2x2, 9 para 3x3): 2 4 5 9 2 1 3 17 7

--- Cálculo del cifrado (C = K * P mod 26) ---
'ata' (P=[0, 19, 0]) -> [24, 12, 11] -> 'yml'
'que' (P=[16, 20, 4]) -> [2, 6, 0] -> 'cga'
'ala' (P=[0, 11, 0]) -> [18, 22, 5] -> 'swf'
'man' (P=[12, 0, 13]) -> [11, 17, 23] -> 'lrx'
'ece' (P=[4, 2, 4]) -> [10, 18, 22] -> 'ksw'
'rxx' (P=[17, 23, 23]) -> [7, 14, 5] -> 'hof'

--- Cálculo del descifrado (P = K^-1 * C mod 26) ---
'yml' (C=[24, 12, 11]) -> [0, 19, 0] -> 'ata'
'cga' (C=[2, 6, 0]) -> [16, 20, 4] -> 'que'
'swf' (C=[18, 22, 5]) -> [0, 11, 0] -> 'ala'
'lrx' (C=[11, 17, 23]) -> [12, 0, 13] -> 'man'
'ksw' (C=[10, 18, 22]) -> [4, 2, 4] -> 'ece'
'hof' (C=[7, 14, 5]) -> [17, 23, 23] -> 'rxx'

------------ Resultados ------------

--> Clave utilizada:           2 4 5 9 2 1 3 17 7
--> Matriz clave K:            [[2, 4, 5], [9, 2, 1], [3, 17, 7]]
--> Determinante (mod 26):     21
--> Matriz inversa K^-1:       [[11, 25, 22], [12, 21, 7], [7, 20, 22]]
--> Mensaje original:          ataque al amanecer
--> Mensaje cifrado:           ymlcgaswflrxkswhof
--> Mensaje descifrado:        ataquealamanecerxx

------------------------------------
```

Figura 2: Prueba 2

### 3.2. Prueba 2: matriz $3 \times 3$ con la clave 2 4 5 9 2 1 3 17 7

El mensaje "ataque al amanecer" tiene 16 letras. Como 16 no es múltiplo de 3, el programa agrega dos `x` al final para completar el último bloque (`rxx`).

Al descifrar se recupera `ataquealamanecerxx`. Las dos `x` del final son el relleno que se agregó para cifrar. Quien recibe el mensaje sabe que son relleno y las ignora.

## 4. Repositorio

El código de esta práctica 05 se puede consultar en el siguiente repositorio:

https://github.com/percivalespi/Criptografia/blob/main/practica5/Algoritmo_Cifrado_Hill.py

## 5. Conclusión

En esta práctica aprendimos cómo se usan las matrices en la criptografía mediante el cifrado de Hill. A diferencia de Playfair, este método utiliza una matriz para cifrar grupos de letras, por lo que al cambiar una letra también cambian las demás letras de su bloque. Algo importante fue entender que el determinante de la matriz debe ser primo relativo con 26 para poder descifrar el mensaje. Calcular la matriz inversa a mano, usando el determinante, la

adjunta y el inverso modular, nos ayudó a comprender mejor el proceso. También vimos que, al descifrar, puede aparecer una letra adicional como la `x` al final del mensaje.